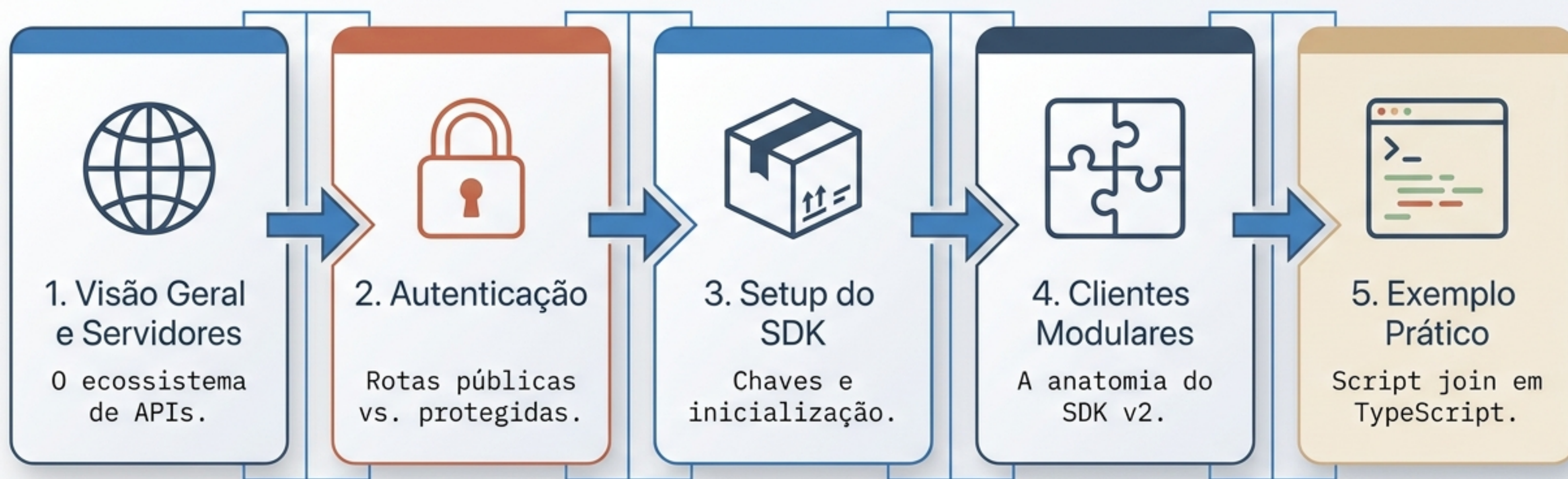


Guia de Início Rápido: API & SDK do Guild.xyz

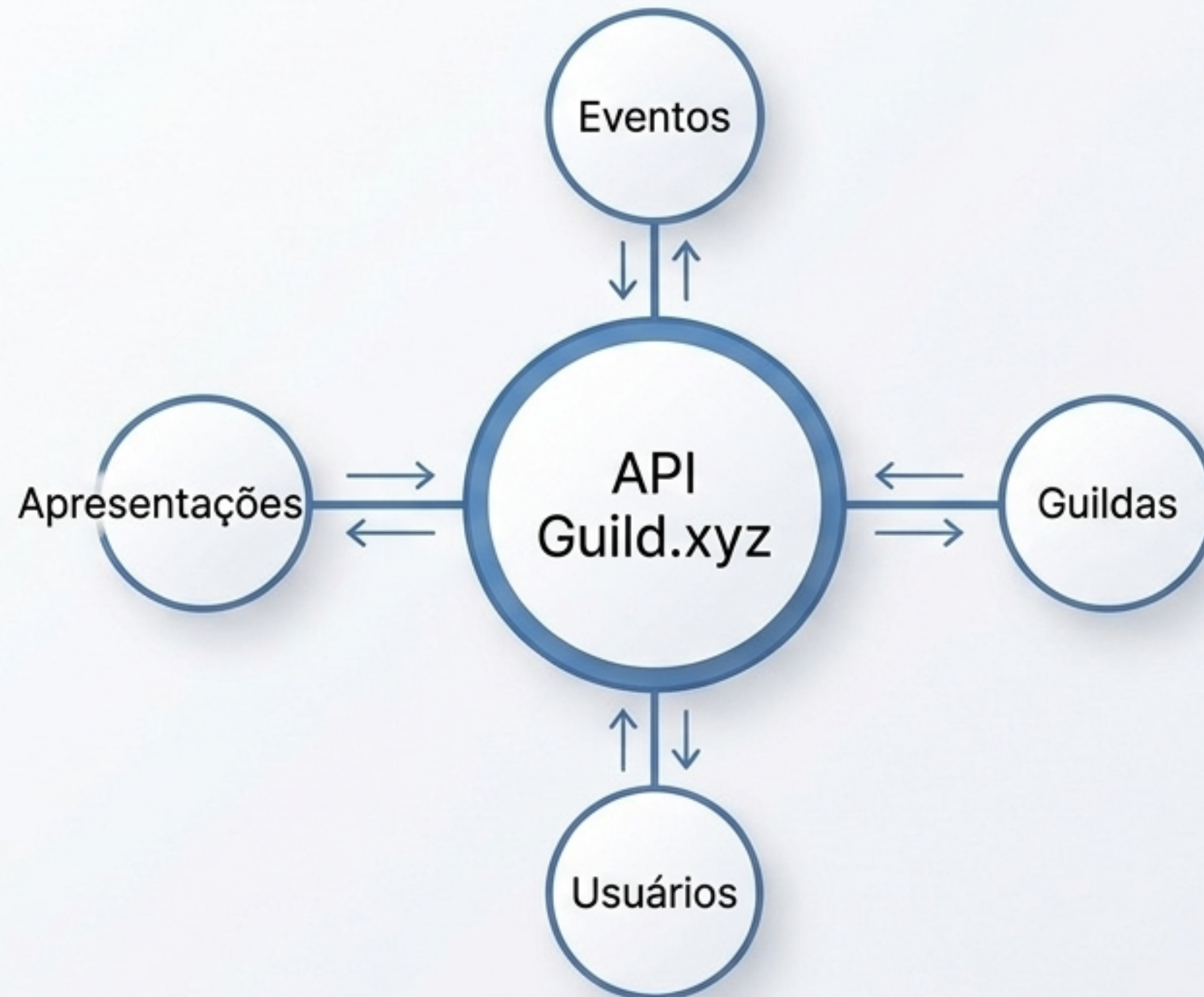
Do Setup Inicial ao Primeiro Ingresso
(Join) Programático.

O Mapa da Documentação



A Camada de Associação da Web3

A API do Guild.xyz abstrai a complexidade on-chain. Ela permite construir integrações robustas que transformam dados de carteiras em permissões programáveis e perfis unificados.



Endereços de Servidores Ativos

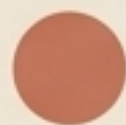


Servidor Principal (Produção)

`https://api.guild.xyz`

Integrações estáveis e SDK base.

Copy



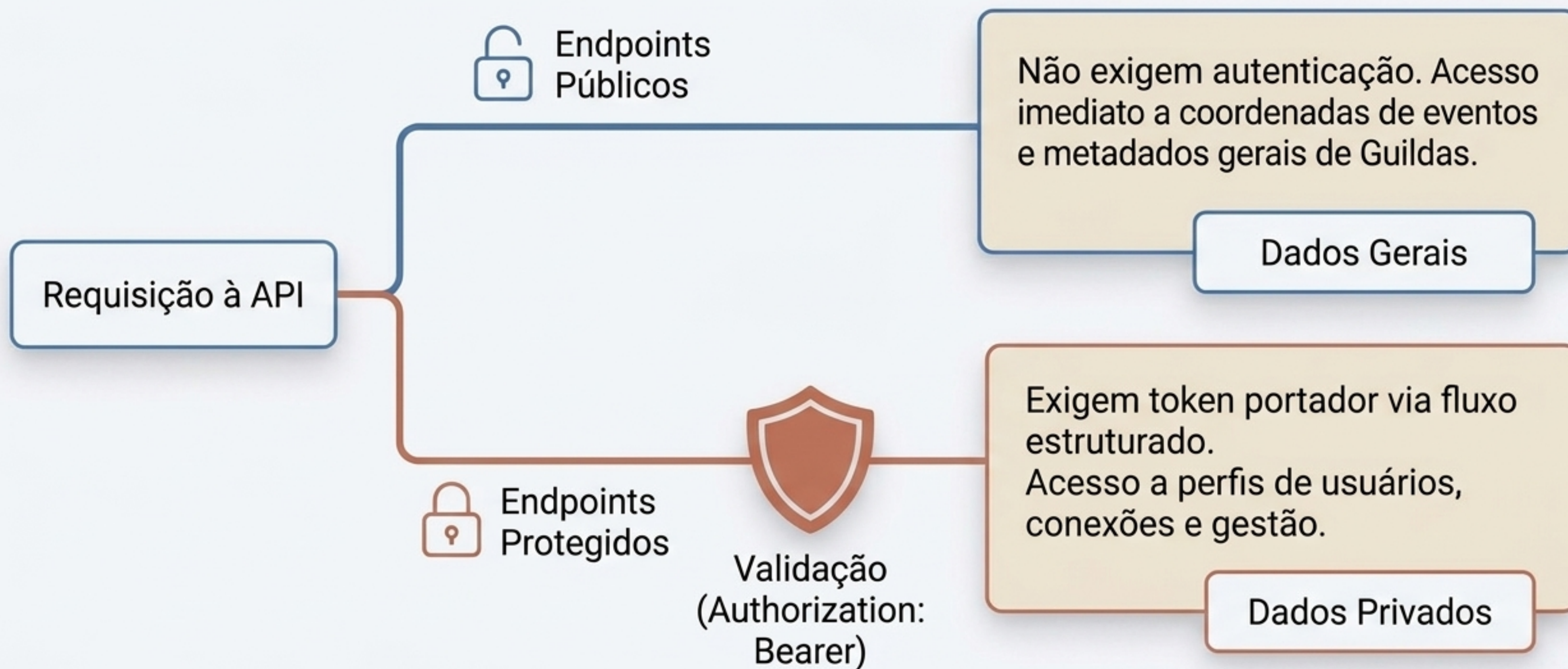
Servidor Alternativo (Next/Alpha)

`https://guild.host/api`

Endpoints em desenvolvimento ativo e fluxos OAuth 2.0 (ex: `/api/oauth/token`).

Copy

Paradigmas de Acesso



Matriz de Rotas e Segurança

Tipo de Rota	Cabeçalho Exigido	Escopo Típico	Casos de Uso
Públicas	Nenhum	N/A	Leitura de Guildas, Eventos.
Protegidos (OAuth 2.0)	Authorization: Bearer <token>	profile:read	Perfis autenticados, claims de email.
Gestão Interna (Sessão)	x-guild-auth-token	Admin	Analytics, CRUD avançado.

Instalação e Inicialização do SDK



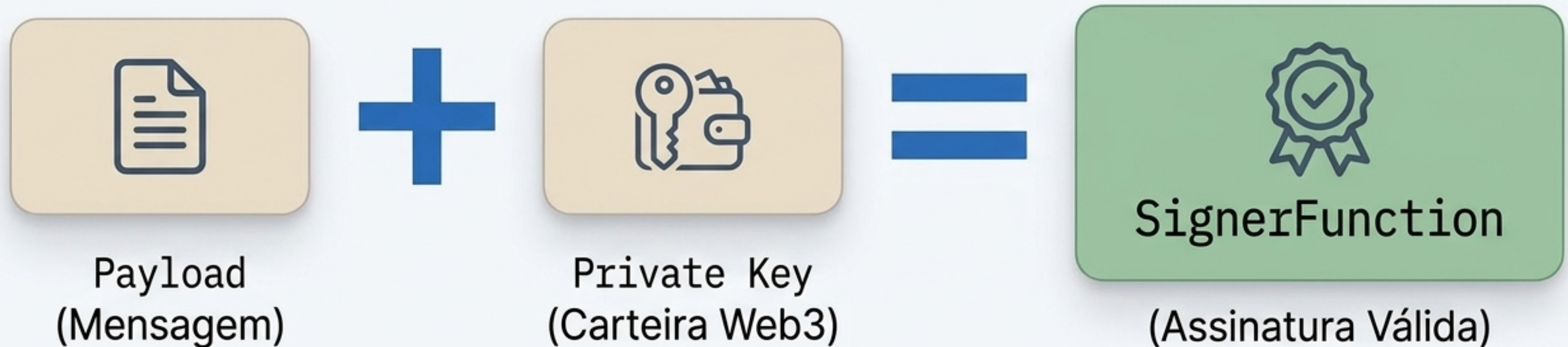
```
> npm i @guildxyz/sdk
```

```
import { createGuildClient, createSigner } from '@guildxyz/sdk';
```

```
// O único parâmetro obrigatório é o nome do seu projeto  
const guildClient = createGuildClient('Meu Projeto Guild');
```


O Paradigma das `SignerFunctions`

No SDK v2, a autenticação programática exige validação criptográfica.



Uma `SignerFunction` recebe um `payload` e gera uma assinatura criptográfica válida para autorizar ações críticas na API, substituindo senhas estáticas.

SignerFunctions na Prática

Backend (Bots / Node.js)

Uso do ethers.js

```
import { ethers } from 'ethers';  
const wallet = new ethers.Wallet('SUA_CHAVE');  
const signer = createSigner.fromEthersWallet(wallet);
```

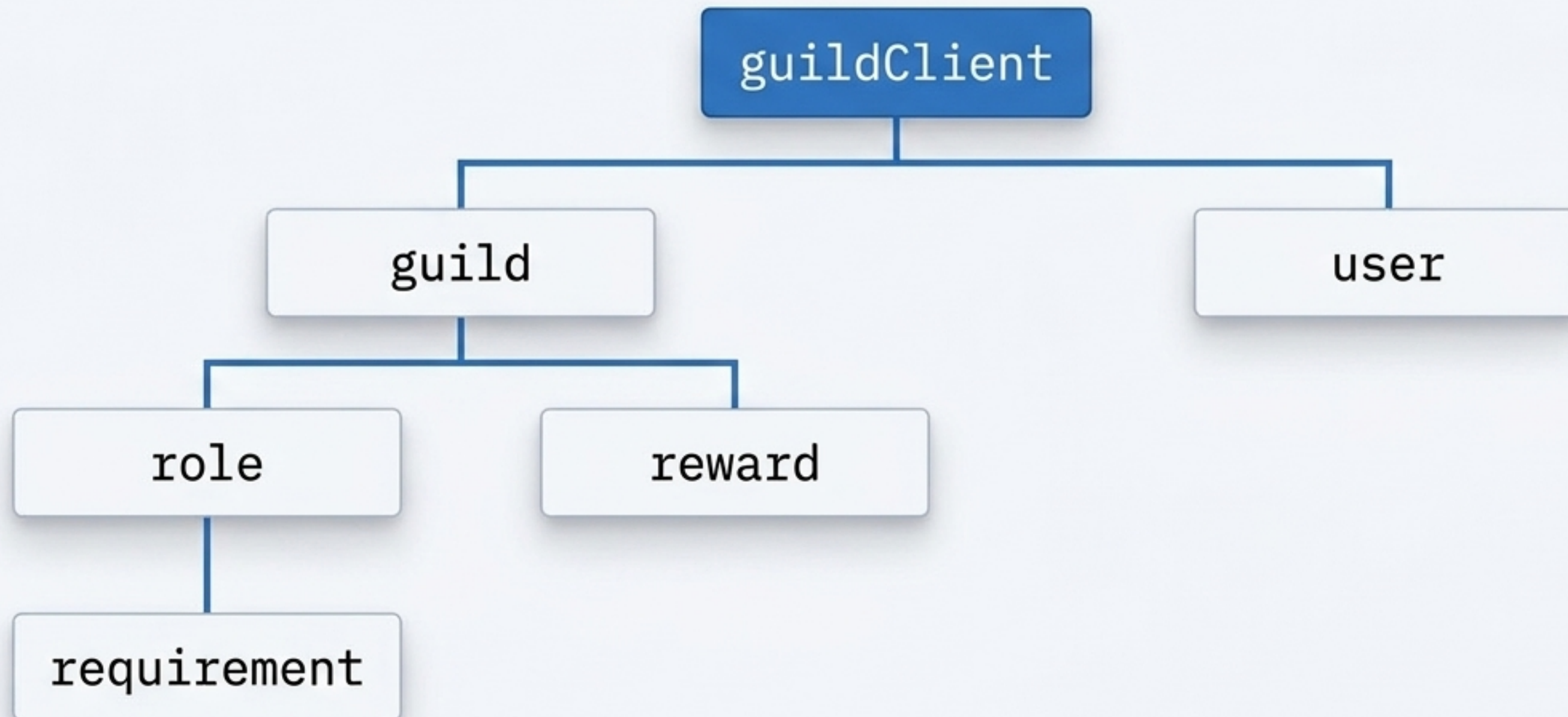
Frontend (React / dApps)

Uso do wagmi

```
import { useAccount, useSignMessage } from 'wagmi';  
const { signMessageAsync } = useSignMessage();  
const { address } = useAccount();  
  
const signer = createSigner.custom(  
  (message) => signMessageAsync({ message }), address  
);
```


A Hierarquia Modular dos Clientes

A estrutura do SDK espelha a arquitetura on-chain. Clientes especializados são instanciados a partir do objeto raiz.



O Catálogo de Subclientes

`guildClient.guild`

Gerencia criação (CRUD) e leitura de guildas.

`guildClient.guild.role`

Controla e define os cargos internos da comunidade.

`...role.requirement`

Configura lógicas de acesso (ex: saldo ERC-20, NFT, Allowlist).

`guildClient.guild.reward`

Associa benefícios off-chain (ex: Cargo no Discord, Telegram).

`guildClient.user`

Resolve perfis, carteiras vinculadas e conexões sociais.

O Fluxo Prático



Código: Criação e Estruturação

```
await guildClient.guild.create({
  name: 'Minha Nova Guilda',
  guildPlatforms: [{ platformName: 'DISCORD', platformGuildId: '123456' }],
  roles: [{
    name: 'Cargo VIP', logic: 'AND',
    requirements: [
      { type: 'ALLOWLIST', data: { addresses: ['0x123...', '0x456...'] } },
      { type: 'ERC20', chain: 'ETHEREUM', address: '0xToken', data: { amount: 1 } }
    ]
  }],
  signerFunction);
```

1 Combina múltiplos requisitos.

2 Define as condições on-chain de entrada.

Código: Ação de Ingresso (Join)

Após a estrutura estar montada, a validação criptográfica é acionada para receber as recompensas.

```
// Ingressando na Guilda se algum cargo for acessível
const joinResult = await guildClient.guild.join(
  guildId,
  signerFunction
);

console.log('Status do Ingresso:', joinResult);
```

```
> Executando script...
> Verificando SignerFunction...
> Status do Ingresso: SUCCESS - Role Granted
```


Complexidade On-Chain, Acesso Programável

O Guild.xyz transforma regras fragmentadas de blockchain em um gateway modular.

